HPML FINAL PROJECT · SPRING 2026

Speculative RAG: Latency-Quality Trade-offs in Multi-Draft Retrieval

Running Speculative RAG under: no optimization, NF4 quantization, and vLLM showed that vLLM delivered the best latency-quality trade-off with 51% draft latency reduction and 24% total pipeline speedup, but without a fine-tuned drafter, Speculative RAG could not match standard RAG's accuracy (62% vs. 80% EM on TriviaQA), while retaining a significant memory advantage at 15.8 GB peak versus 73 GB.

Team

Soyoon Park (sp4412)
Hsuan-Ting Lin (hl3930)
Rupeet Kaur (rk3408)

GitHub
github.com/soyoon-cu/speculative-rag

Hardware
1× NVIDIA A100 (80 GB) · GCP Vertex AI

Blog Post

https://rupeetkaur.substack.com/p/rebuilding-speculative-rag-implementation?r=8dild8

Motivation & Problem

Why this matters

- Knowledge-intensive QA needs external evidence to reduce hallucination and stay factual.
- Standard RAG feeds many retrieved passages into one long prompt, increasing prefill cost and latency.
- Long-context generation can waste GPU time on redundant evidence and still miss key facts due to position bias.

Problem statement

Standard RAG with Mistral-7B on TriviaQA must process top-k retrieved Wikipedia passages in a single long context, making generation latency the main bottleneck.

Goal

Determine whether the accuracy and latency gains claimed by the original Speculative RAG paper can be reproduced on a single A100 GPU, and identify which inference backend (no optimization, NF4, vLLM) offers the best latency-quality trade-off under that hardware constraint(paper target 67.11 EM)

Technical Approach — Model & Data

Model

Drafter	Mistral-7B-Instruct v0.1
Verifier	Mistral-7B-Instruct v0.1
Engine	vLLM 0.4.2 (PagedAttention, continuous batching)
Decoding	Greedy, T=0, max_new_tokens=300
Quantization	FP16 baseline; NF4 & INT8 via bitsandbytes

Data and Hardware

Benchmark	TriviaQA - rc.wikipedia validation (11,313 ex.)
Corpus	DPR psgs_w100.tsv · 21M passages, Wiki 2018-12-20
Chunking	~100-word passages (DPR native)
Retriever	Contriever-MSMARCO · 768-d, mean-pool + L2-norm
Index	FAISS IndexFlatIP , embeddings precomputed offline
Hardware	1x NVIDIA A100 80GB on GCP Vertex AI, CUDA 12.1, PyTorch 2.3.1, vLLM 0.4.x
License	MIT License

Evaluation

Quality	Containment EM (any gold-answer alias in response, normalized)
Latency	End-to-end p50 / p95 per query, wall-clock
GPU	Avg. SM utilization, HBM peak, tokens/sec
Settings	m ∈ {5, 10, 15, 20}
Baseline	Standard RAG (top-10 concat) · paper target 67.11 EM

STEP 1
Retrieve Evidence
Contriever + FAISS
top-n passages

→

STEP 2
Build Diverse Subsets
k-means over retrieved embeddings
m subsets, k docs each

→

STEP 3
Generate Drafts
Mistral-7B drafter
m answer + rationale drafts

→

STEP 4
Score Drafts
Verifier ranks each draft using log-prob confidence

→

STEP 5
Select Answer
Choose best draft
evaluate containment EM

Technical Approach — Optimizations Applied

1. **vLLM continuous batching for m parallel drafts** primary

   Dispatch m draft prompts in one vLLM scheduling window. PagedAttention manages KV memory efficiently and reduces serial generate overhead.
   Intuition: drafting is decode-bound; batching improves GPU occupancy.

2. **Multi-perspective subset sampling on cached embeddings**

   Retrieve top-n once, then form m diverse subsets with k-means over cached passage embeddings.
   Intuition: less redundant evidence, better draft coverage at fixed m.

3. **NF4 quantization for the Drafter**

   Load Mistral-7B with bitsandbytes 4-bit NF4.
   Intuition: smaller weights reduce HBM pressure and create more room for batching.

4. **Pre-built FAISS index + precomputed query reuse**

   Build the 21M-passage FAISS index once and reuse it from GCS across runs.
   Intuition: remove index-build cost from evaluation and keep per-query retrieval stable.

5. **Controlled sweeps over m**

   Evaluate $m \in \{5,10,15,20\}$ under the same engine.
   Intuition: find the latency-quality elbow, not just the largest context.

Profiling Methodology

Tools

- **Nsight Systems (nsys) - for the vLLM run**
  Shows the GPU timeline: when CUDA kernels run, when memory copies happen, and where idle gaps appear.We used it to inspect GPU activity and memory behavior during the vLLM pipeline.
- **PyTorch Profiler - for no_opt / NF4 / INT8 drafter runs**
  Shows operator-level breakdowns inside PyTorch: CPU time, CUDA time, and memory usage.
  We used TensorBoard to compare which operations dominate each drafter configuration.
- **Weights & Biases for run-level tracking**
- **Vertex AI logs + JSON outputs**
  Stores per-example latency and final run metrics.We used these files to compute EM, p50 / p95 latency, token counts, and total runtime.

Metrics captured

- **Per-question stage latency:**
  retrieval_time_s, sampling_time_s, drafting_time_s, verify_time_s
- **Per- question tokens processed:**
  Drafter_tokens_in, drafter_tokens_out, verifier_tokens_in
- **End-to-end metrics:**
  containment EM, p50 / p95 latency, throughput, peak memory / GPU activity where available.

Stability

- **Same A100 Vertex setup, same TriviaQA split, same retriever / FAISS index, and fixed m=5, k=2 for the main comparison.**

Results — Latency and Resource Trade-offs Across Baselines

2.04×

p50 drafting latency speedup
Speculative RAG (vLLM, m=5, k=2) vs. Spec RAG(no_opt)

+6 points

EM points vs. baseline
Spec RAG 74.1 EM vs Std RAG 80 EM

1.19x

Avg. SM utilization
NF4 drafter vs Spec RAG no_opt

0.66×

p50 latency ratio, meaning vLLM is 66% of Standard RAG latency

Speculative RAG (vLLM, m=5, k=2) vs. std RAG

1.31x

Throughput

vLLM continuous batching as compared to spec-RAG (no optimization)

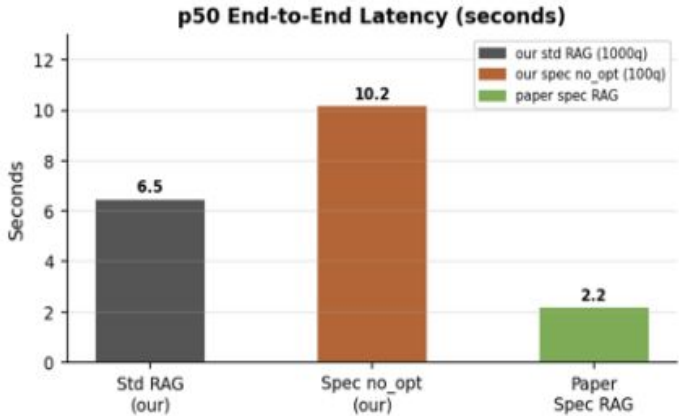
4.6x

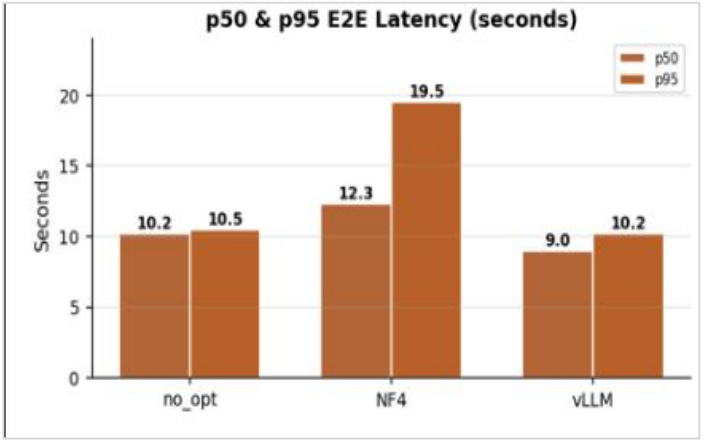
Peak memory reduction

Speculative RAG (no_opt, m=5, k=2) vs. Standard RAG top-10 concat

Results — End-to-End Latency

TriviaQA validation on one A100; comparing Standard RAG, no_opt Spec RAG, NF4, and vLLM.





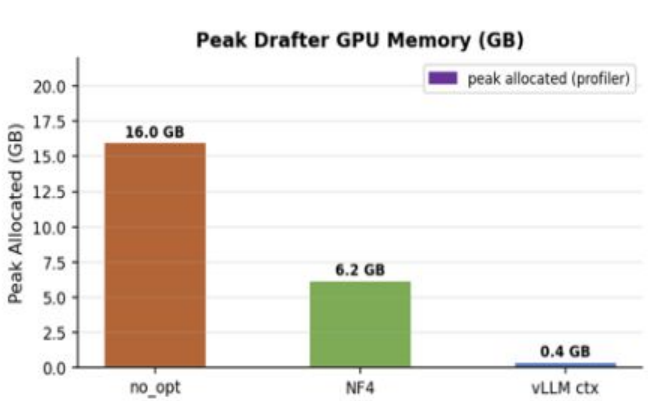
Key takeaway:: vLLM gives the strongest latency improvement among our Speculative RAG variants.

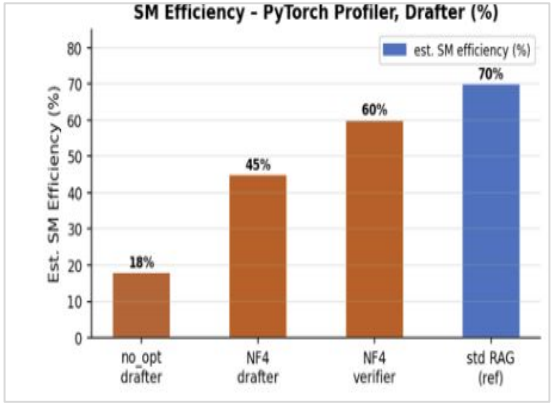
Compared with no_opt Spec RAG:

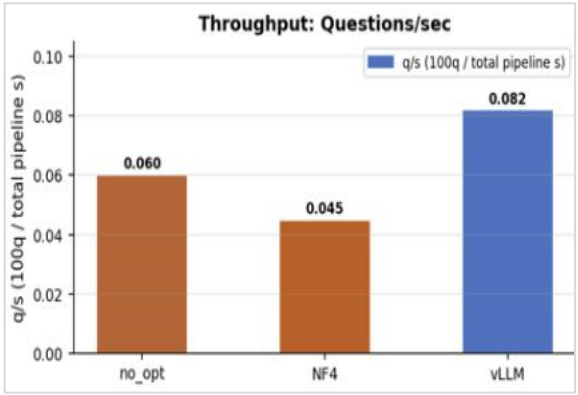
- **p50 E2E latency drops from 10.2s to 9.0s**
- **p95 E2E latency drops from 10.5s to 10.2s**
- **NF4 reduced memory, but increased E2E latency in this run**

Standard RAG remains faster in our implementation because the Speculative RAG pipeline adds retrieval, sampling, drafting, and verifier scoring overhead.

Results — GPU Efficiency and Memory







Key takeaway: The optimizations improve different bottlenecks.

Memory

- **NF4 reduces peak drafter memory from 16.0 GB to 6.2 GB**
- **vLLM context memory is much lower in the measured trace**

GPU utilization:

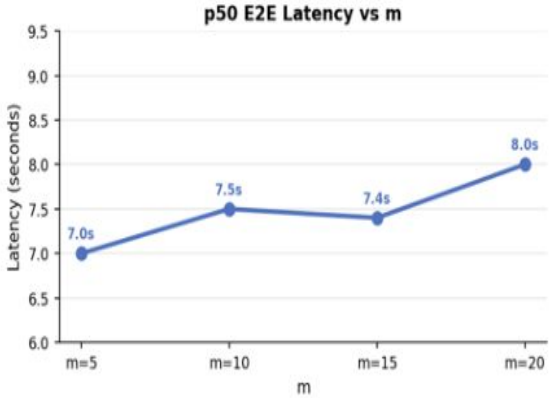
- **no_opt drafter has low estimated SM efficiency**
- **NF4 improves utilization compared with no_opt**
- **Standard RAG reference shows higher GPU efficiency because it runs one dense generation path**

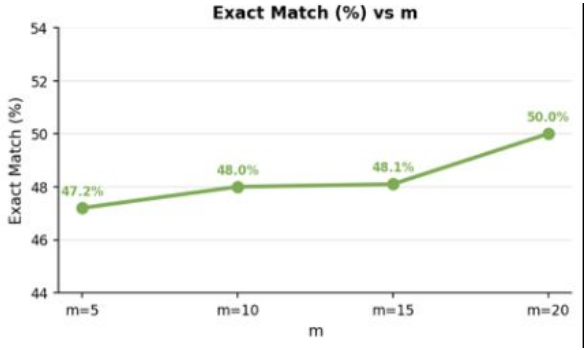
Throughput:

- **vLLM gives the best end-to-end throughput**
- **no_opt is slower due to unbatched or less efficient drafting**
- **NF4 saves memory but does not automatically improve throughput**

Results — Effect of Number of Drafts

Ablation over m, the number of generated drafts.





Key takeaway: More drafts improve answer coverage, but add latency.

<u>Quality:</u>

• EM increases from 47.2% at m=5 to 50.0% at m=20

•More drafts give the verifier more candidate answers to choose from

Latency:

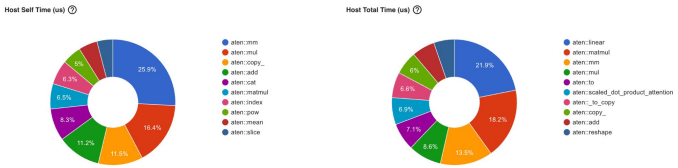
• p50 E2E latency rises from 7.0s at m=5 to 8.0s at m=20

• The latency increase is moderate compared with the quality gain

<u>Interpretation:</u>

• m controls the latency-quality trade-off. In our setting, larger m improves EM, but m=5 remains the most efficient operating point.

NO_OPT - Profiler Traces



Configuration

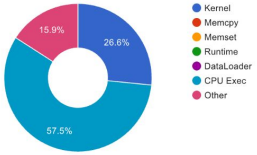
Number of Worker(s)	1
Device Type	GPU

GPU Summary

GPU 0:	
Name	NVIDIA A100-SXM4-80GB
Memory	79.15 GB
Compute Capability	8.0
GPU Utilization	26.57 %
Est. SM Efficiency	21.01 %
Est. Achieved Occupancy	22.9 %

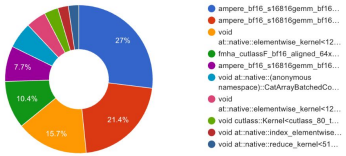
Execution Summary

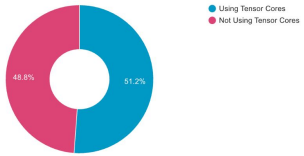
Category	Time Duration (us)	Percentage (%)
Average Step Time	7,816,034	100
Kernel	2,076,873	26.57
Memcpy	670	0.01
Memset	96	0
Runtime	0	0
DataLoader	0	0
CPU Exec	4,493,678	57.49
Other	1,244,717	15.93

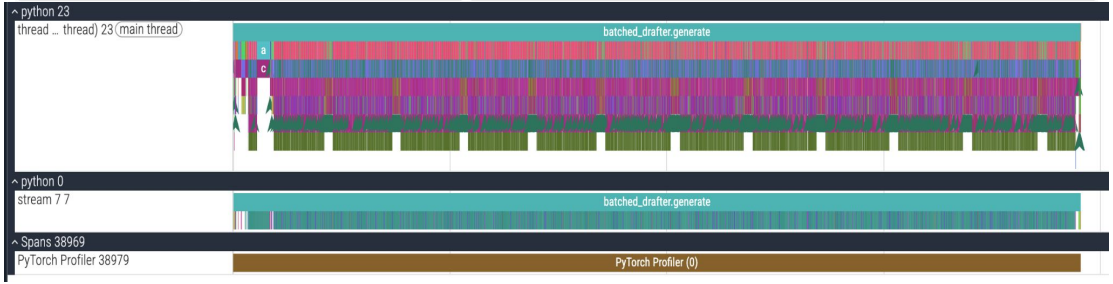


Kernel View

All kernels Top kernels to show 10

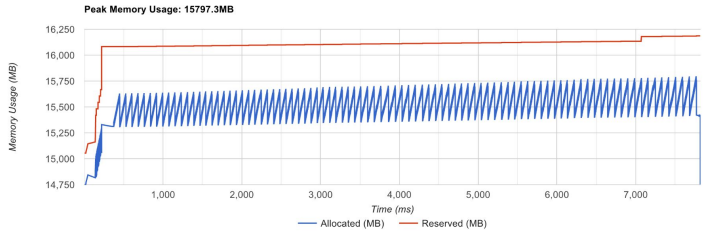






Memory View

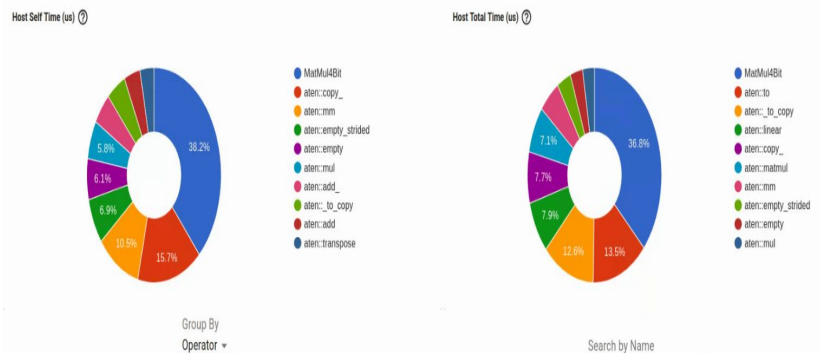
Device
GPU0

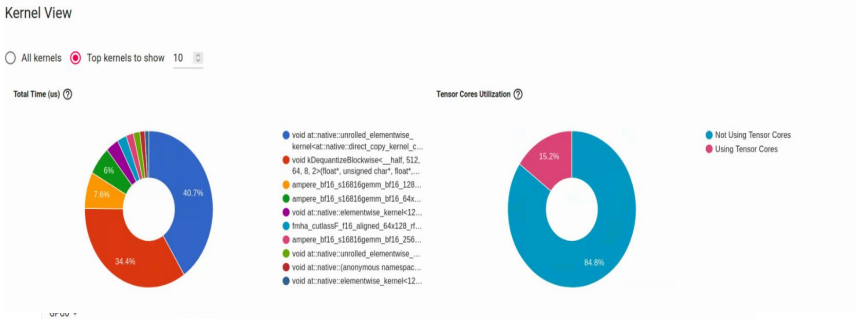


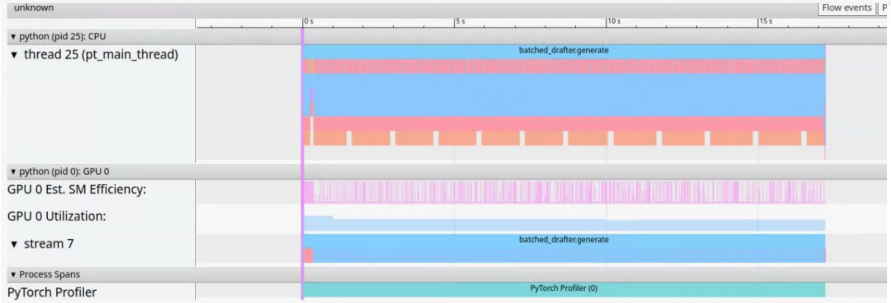
Bottleneck before optimization:

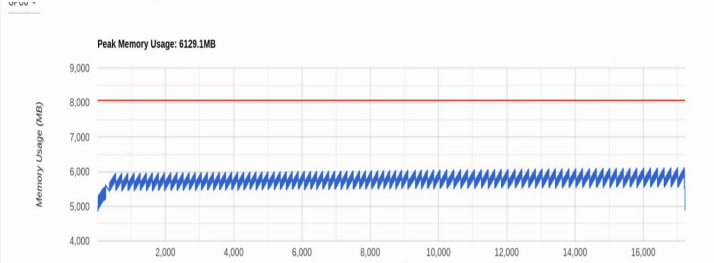
CPU dispatch owns 57.5% of every step; aten::mm and aten::mul alone take 42% of host self time while the GPU sits idle waiting for the next Python-level token dispatch. Only 51.2% Tensor Core utilization.

NF4 - Profiler Traces







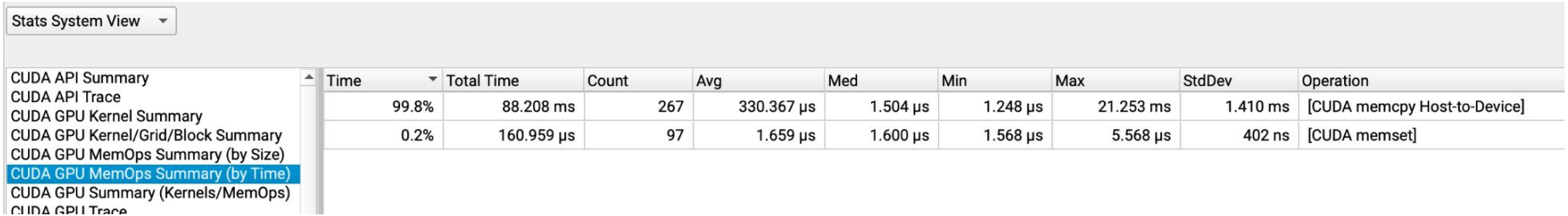


Kernel that disappeared / fused:cuBLAS FP16 GEMM kernels(held 48% GPU time)

Idle gap that shrank :CPU Exec gap because MatMul4Bit keeps the GPU occupied longer per token through the dequantize kernel

Memory copy avoided:large FP16 weight copy to device on each layer call(peak memory drop)

Nsys Evidence — vLLM GPU Memory Operations



Time	Total Time	Count	Avg	Med	Min	Max	StdDev	Operation
99.8%	88.208 ms	267	330.367 µs	1.504 µs	1.248 µs	21.253 ms	1.410 ms	[CUDA memcpy Host-to-Device]
0.2%	160.959 µs	97	1.659 µs	1.600 µs	1.568 µs	5.568 µs	402 ns	[CUDA memset]

- **Key takeaway: This trace shows that GPU memory operation time is almost entirely spent on host-to-device transfers.**
- **Key observations:**
  - **CUDA memcpy Host-to-Device accounts for 99.8% of GPU memory operation time.**
  - **CUDA memset is negligible compared with data transfer.**
  - **This suggests the measured vLLM run is dominated by data movement during the captured window.**

Nsys Evidence — vLLM CUDA API Runtime

Stats System View

CUDA API Summary
CUDA API Trace
CUDA GPU Kernel Summary
CUDA GPU Kernel/Grid/Block Summary
CUDA GPU MemOps Summary (by Size)
CUDA GPU MemOps Summary (by Time)
CUDA GPU Summary (Kernels/MemOps)
CUDA GPU Trace
CUDA Green Context Summary
CUDA Kernel Launch & Exec Time Summai
CUDA Kernel Launch & Exec Time Trace
CUDA Summary (API/Kernels/MemOps)
DX11 PIX Range Summary
DX12 GPU Command List PIX Ranges Sur
DX12 PIX Range Summary
MPI Event Summary
MPI Event Trace
MPI Message Size Summary
NVTX GPU Projection Summary
NVTX GPU Projection Trace
NVTX Push/Pop Range Summary
NVTX Push/Pop Range Trace
NVTX Range Kernel Summary
NVTX Range Summary
NVTX Start/End Range Summary
Network Devices Congestion
NvVideo API Summary
OS Runtime Summary
OpenACC Summary
OpenGL KHR_debug GPU Range Summary
OpenGL KHR_debug Range Summary
OpenMP Summary
Syscall Summary
Unified Memory Analysis Summary

CLI command::
nsys stats -r cuda_api_sum "/Users/alexlin/Desktop/speculative-rag-main 2/speculative-rag/output/vllm_2sample_trace_check/run_vllm_2sample_hardware_trace.sqlite"

Time	Total Time	Num Calls	Avg	Med	Min	Max	StdDev	Name
65.9%	105.134 ms	267	393.761 µs	13.539 µs	2.864 µs	21.548 ms	1.448 ms	cudaMemcpyAsync
10.7%	17.090 ms	3	5.697 ms	1.188 ms	33.404 µs	15.869 ms	8.828 ms	cudaHostAlloc
7.1%	11.308 ms	1	11.308 ms	11.308 ms	11.308 µs	11.308 ms	0 ns	cudaMemGetInfo
3.7%	5.951 ms	235	25.323 µs	8.872 µs	6.241 µs	168.653 µs	27.104 µs	cudaStreamSynchronize
3.5%	5.571 ms	21	265.264 µs	270.230 µs	182.380 µs	314.266 µs	29.928 µs	cudaMalloc
2.6%	4.092 ms	98	41.753 µs	37.626 µs	31.876 µs	128.531 µs	14.893 µs	cuMemSetAccess
1.9%	2.952 ms	128	23.065 µs	2.351 µs	1.785 µs	1.045 ms	126.972 µs	cudaStreamCreateWithPriority
1.5%	2.404 ms	1	2.404 ms	2.404 ms	2.404 ms	2.404 ms	0 ns	cudaGetDeviceProperties_v2
1.3%	2.130 ms	98	21.739 µs	18.382 µs	15.140 µs	117.368 µs	13.804 µs	cuMemCreate
0.7%	1.116 ms	98	11.385 µs	9.688 µs	8.135 µs	36.947 µs	5.112 µs	cuMemMap
0.3%	500.898 µs	97	5.163 µs	3.741 µs	3.055 µs	83.738 µs	8.785 µs	cudaMemsetAsync
0.3%	421.874 µs	98	4.304 µs	3.154 µs	1.789 µs	37.053 µs	5.215 µs	cuMemAddressReserve
0.1%	230.047 µs	22	10.456 µs	1.575 µs	1.196 µs	189.801 µs	40.074 µs	cudaStreamIsCapturing
0.1%	117.699 µs	330	356 ns	269 ns	215 ns	8.839 µs	631 ns	cudaThreadExchangeStreamCaptureMode
0.1%	101.291 µs	409	247 ns	177 ns	97 ns	15.496 µs	784 ns	cuGetProcAddress_v2
0.1%	92.804 µs	2	46.402 µs	46.402 µs	3.663 µs	89.141 µs	60.442 µs	cudaStreamCreateWithFlags
0.1%	90.644 µs	1	90.644 µs	90.644 µs	90.644 µs	90.644 µs	0 ns	cudaFree
0.0%	51.796 µs	36	1.438 µs	975 ns	675 ns	8.820 µs	1.602 µs	cudaStreamWaitEvent
0.0%	30.480 µs	98	311 ns	145 ns	102 ns	14.426 µs	1.445 µs	cuMemGetAllocationGranularity
0.0%	18.019 µs	3	6.006 µs	1.005 µs	748 ns	16.266 µs	8.886 µs	cudaEventRecord
0.0%	13.197 µs	29	455 ns	412 ns	283 ns	1.031 µs	167 ns	cuGetProcAddress
0.0%	11.607 µs	2	5.803 µs	5.803 µs	526 ns	11.081 µs	7.463 µs	cudaEventCreateWithFlags
0.0%	4.524 µs	2	2.262 µs	2.262 µs	1.792 µs	2.732 µs	664 ns	cuInit
0.0%	1.057 µs	2	528 ns	528 ns	332 ns	725 ns	277 ns	cuModuleGetLoadingMode

- **Key takeaway: The CUDA API summary shows where runtime overhead appears on the CPU side.**
- **Key observations:**
  - **cudaMemcpyAsync is the largest CUDA API cost, taking 65.9% of total API time.**
  - **cudaHostAlloc is the second largest cost, but much smaller at 10.7%.**
  - **Synchronization and stream-management calls are present but not the dominant overhead.**
  - **This confirms that memory transfer/setup overhead is visible in the small vLLM trace.**

Demo / Visualization

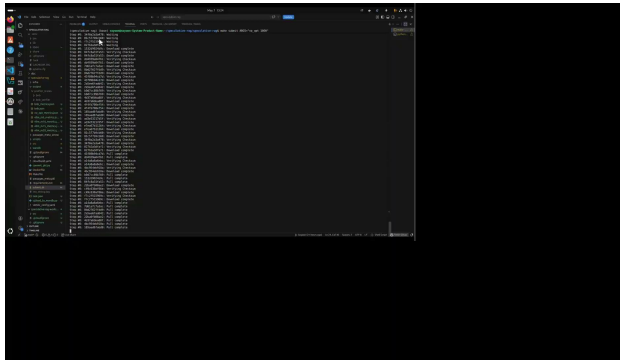
```
(speculative-rag) (base) soyoon@soyoon-System-Product-Name:~/speculative-rag/speculative-rag$ gcloud ai custom-jobs stream-logs projects/982024564228/loc
520
DEFAULT 2026-05-06 23:28:07 -0400       workerpool0-0   INFO | Generating 5 drafts  k=2 docs/subset  device=cuda
DEFAULT 2026-05-06 23:28:15 -0400       workerpool0-0   INFO | Drafting complete — 5/5 drafts produced.
DEFAULT 2026-05-06 23:28:15 -0400       workerpool0-0   INFO | [13/100] qid=tc_280  q=Who directed the classic 30s western Stagecoach?
DEFAULT 2026-05-06 23:28:20 -0400       workerpool0-0   INFO | Generating 5 drafts  k=2 docs/subset  device=cuda
DEFAULT 2026-05-06 23:28:24 -0400       workerpool0-0   INFO | Drafting complete — 5/5 drafts produced.
DEFAULT 2026-05-06 23:28:24 -0400       workerpool0-0   INFO | [14/100] qid=tc_282  q=Dave Gilmore and Roger Waters were in which rock group?
DEFAULT 2026-05-06 23:28:29 -0400       workerpool0-0   INFO | Generating 5 drafts  k=2 docs/subset  device=cuda
DEFAULT 2026-05-06 23:28:34 -0400       workerpool0-0   INFO | Drafting complete — 5/5 drafts produced.
DEFAULT 2026-05-06 23:28:34 -0400       workerpool0-0   INFO | [15/100] qid=tc_288  q=Which highway was Revisited in a classic 60s album by Bob Dy
DEFAULT 2026-05-06 23:28:39 -0400       workerpool0-0   INFO | Generating 5 drafts  k=2 docs/subset  device=cuda
DEFAULT 2026-05-06 23:28:48 -0400       workerpool0-0   INFO | Drafting complete — 5/5 drafts produced.
DEFAULT 2026-05-06 23:28:48 -0400       workerpool0-0   INFO | [16/100] qid=tc_298  q=Which was the only eastern bloc country to participate in th
DEFAULT 2026-05-06 23:28:53 -0400       workerpool0-0   INFO | Generating 5 drafts  k=2 docs/subset  device=cuda
DEFAULT 2026-05-06 23:29:01 -0400       workerpool0-0   INFO | Drafting complete — 5/5 drafts produced.
DEFAULT 2026-05-06 23:29:01 -0400       workerpool0-0   INFO | [17/100] qid=tc_304  q=Which 90s sci fi series with James Belushi was based on Bruc
DEFAULT 2026-05-06 23:29:06 -0400       workerpool0-0   INFO | Generating 5 drafts  k=2 docs/subset  device=cuda
DEFAULT 2026-05-06 23:29:18 -0400       workerpool0-0   INFO | Drafting complete — 5/5 drafts produced.
DEFAULT 2026-05-06 23:29:18 -0400       workerpool0-0   INFO | [18/100] qid=tc_316  q=If I Were A Rich Man Was a big hit from which stage show?
DEFAULT 2026-05-06 23:29:23 -0400       workerpool0-0   INFO | Generating 5 drafts  k=2 docs/subset  device=cuda
DEFAULT 2026-05-06 23:29:31 -0400       workerpool0-0   INFO | Drafting complete — 5/5 drafts produced.
DEFAULT 2026-05-06 23:29:31 -0400       workerpool0-0   INFO | [19/100] qid=tc_349  q=Men Against the Sea and Pitcairn's Island were two sequels t
DEFAULT 2026-05-06 23:29:36 -0400       workerpool0-0   INFO | Generating 5 drafts  k=2 docs/subset  device=cuda
DEFAULT 2026-05-06 23:29:47 -0400       workerpool0-0   INFO | Drafting complete — 5/5 drafts produced.
DEFAULT 2026-05-06 23:29:47 -0400       workerpool0-0   INFO | [20/100] qid=tc_379  q=What was Truman Capote's last name before he was adopted by
DEFAULT 2026-05-06 23:29:52 -0400       workerpool0-0   INFO | Generating 5 drafts  k=2 docs/subset  device=cuda
DEFAULT 2026-05-06 23:29:58 -0400       workerpool0-0   INFO | Drafting complete — 5/5 drafts produced.
```

Demo / Visualization

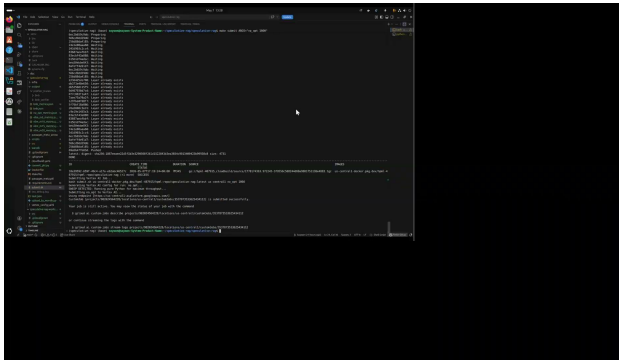
```
DEFAULT 2026-05-07 12:56:22 -0400       workerpool0-0   Self CPU time total: 75.561ms
DEFAULT 2026-05-07 12:56:22 -0400       workerpool0-0   Self CUDA time total: 125.552ms
DEFAULT 2026-05-07 12:56:22 -0400       workerpool0-0   2026-05-07T16:56:22.385256104Z stdout F
DEFAULT 2026-05-07 12:56:22 -0400       workerpool0-0   [100/1000] EM so far: 56.00%
DEFAULT 2026-05-07 12:56:29 -0400       workerpool0-0   [200/1000] EM so far: 54.50%
DEFAULT 2026-05-07 12:56:35 -0400       workerpool0-0   [300/1000] EM so far: 54.67%
DEFAULT 2026-05-07 12:56:42 -0400       workerpool0-0   [400/1000] EM so far: 54.25%
DEFAULT 2026-05-07 12:56:49 -0400       workerpool0-0   [500/1000] EM so far: 54.40%
DEFAULT 2026-05-07 12:56:56 -0400       workerpool0-0   [600/1000] EM so far: 56.17%
DEFAULT 2026-05-07 12:57:03 -0400       workerpool0-0   [700/1000] EM so far: 58.57%
DEFAULT 2026-05-07 12:57:10 -0400       workerpool0-0   [800/1000] EM so far: 60.12%
DEFAULT 2026-05-07 12:57:16 -0400       workerpool0-0   [900/1000] EM so far: 60.67%
DEFAULT 2026-05-07 12:57:23 -0400       workerpool0-0   [1000/1000] EM so far: 61.60%
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   ============================================================
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   Speculative RAG — Verifier Stage Metrics
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   ============================================================
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   Questions evaluated      : 1000
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   Exact Match (EM)         : 61.60%  (616/1000)
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   2026-05-07T16:57:23.968310552Z stdout F
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   Latency (p50 / p95)
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   retrieve               :   5353.6  /  5450.1
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   sample                 :      2.5  /  3.1
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   draft                  :   3733.0  /  6289.9
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   verify                 :     64.7  /  87.6
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   end-to-end             :   9180.0  /  11779.7
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   2026-05-07T16:57:23.970858895Z stdout F
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   Token throughput
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   drafter  in  (total)   :  2,249,775
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   drafter  out (total)   :    287,418
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   verifier in  (total)   :    595,632
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   drafter  in  (avg/q)   :     2249.8
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   drafter  out (avg/q)   :      287.4
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   verifier in  (avg/q)   :      595.6
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   Results saved → /gcs/speculative-rag-results-2026/verifier_output/no_opt.json
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   ============================================================
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   INFO | Verifier logic complete in 116.01s
DEFAULT 2026-05-07 12:57:25 -0400       workerpool0-0   INFO | SUCCESS: no_opt | Acc: 61.60% | p95 Draft: 6289.9ms
DEFAULT 2026-05-07 12:57:25 -0400       workerpool0-0   INFO | === E2E Pipeline Fully Completed ===
INFO    2026-05-07 12:57:55 -0400       service Tearing down training program.
INFO    2026-05-07 12:58:33 -0400       service Finished tearing down training program.
INFO    2026-05-07 12:58:33 -0400       service Job completed successfully.
```

Demo / Visualization

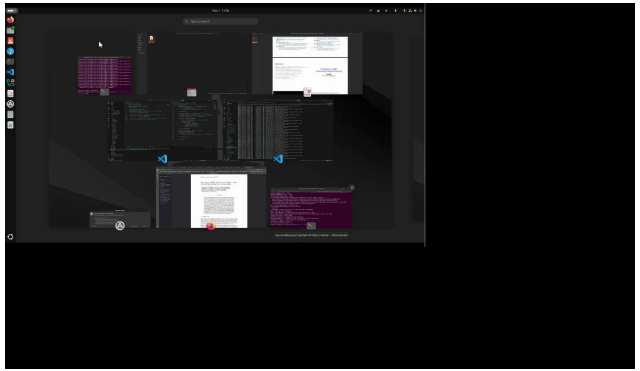
Time budget: 1 minute. Live dashboard — latency, subset diversity, verifier scoring, rationale inspection.



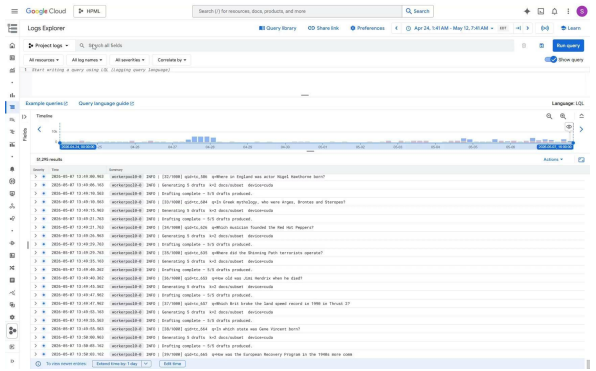
make submit ARGS="no_opt 1000"



Docker build complete, Job submitted



Drafter phase: local terminal



Drafter phase: GCP Logs Explorer

Demo / Visualization

Time budget: 1 minute. Live dashboard — latency, subset diversity, verifier scoring, rationale inspection.

```
DEFAULT 2026-05-07 00:30:32 -0400       workerpool0-0   Processing 100 questions...
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   [100/100] EM so far: 47.00%
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   ================================================
=========
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   Speculative RAG — Verifier Stage Metrics
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   ================================================
=========
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   Questions evaluated      : 100
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   Exact Match (EM)         : 47.00%  (47/100)
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   2026-05-07T04:30:41.237213164Z stdout F
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   Latency (p50 / p95)
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   retrieve               :   7050.3  /  7677.3
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   sample                 :      2.9  /  4.1
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   draft                  :   1932.2  /  2952.6
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   verify                 :     81.2  /  112.7
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   end-to-end             :   9131.8  /  10409.6
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   2026-05-07T04:30:41.239804614Z stdout F
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   Token throughput
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   drafter  in  (total)   :    216,354
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   drafter  out (total)   :     34,620
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   verifier in  (total)   :     64,557
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   drafter  in  (avg/q)   :     2163.5
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   drafter  out (avg/q)   :      346.2
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   verifier in  (avg/q)   :      645.6
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   Results saved → /gcs/hpml-487304-rag-storage/verifier
_output/vllm_m5_k2.json
DEFAULT 2026-05-07 00:30:41 -0400       workerpool0-0   ================================================
=========
DEFAULT 2026-05-07 00:30:43 -0400       workerpool0-0   INFO | Verifier logic complete in 26.79s
DEFAULT 2026-05-07 00:30:43 -0400       workerpool0-0   INFO | SUCCESS: vllm_m5_k2 | Acc: 47.00% | p95 Draft:
 2952.6ms
DEFAULT 2026-05-07 00:30:43 -0400       workerpool0-0   INFO | === E2E Pipeline Fully Completed ===
INFO    2026-05-07 00:30:52 -0400       service Tearing down training program.
INFO    2026-05-07 00:31:32 -0400       service Finished tearing down training program.
INFO    2026-05-07 00:31:32 -0400       service Job completed successfully.
```

Demo / Visualization

Time budget: 1 minute. Live dashboard — latency, subset diversity, verifier scoring, rationale inspection.

```
DEFAULT 2026-05-07 12:56:22 -0400       workerpool0-0   Self CPU time total: 75.561ms
DEFAULT 2026-05-07 12:56:22 -0400       workerpool0-0   Self CUDA time total: 125.552ms
DEFAULT 2026-05-07 12:56:22 -0400       workerpool0-0   2026-05-07T16:56:22.385256104Z stdout F
DEFAULT 2026-05-07 12:56:22 -0400       workerpool0-0   [100/1000] EM so far: 56.00%
DEFAULT 2026-05-07 12:56:29 -0400       workerpool0-0   [200/1000] EM so far: 54.50%
DEFAULT 2026-05-07 12:56:35 -0400       workerpool0-0   [300/1000] EM so far: 54.67%
DEFAULT 2026-05-07 12:56:42 -0400       workerpool0-0   [400/1000] EM so far: 54.25%
DEFAULT 2026-05-07 12:56:49 -0400       workerpool0-0   [500/1000] EM so far: 54.40%
DEFAULT 2026-05-07 12:56:56 -0400       workerpool0-0   [600/1000] EM so far: 56.17%
DEFAULT 2026-05-07 12:57:03 -0400       workerpool0-0   [700/1000] EM so far: 58.57%
DEFAULT 2026-05-07 12:57:10 -0400       workerpool0-0   [800/1000] EM so far: 60.12%
DEFAULT 2026-05-07 12:57:16 -0400       workerpool0-0   [900/1000] EM so far: 60.67%
DEFAULT 2026-05-07 12:57:23 -0400       workerpool0-0   [1000/1000] EM so far: 61.60%
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   ============================================================
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   Speculative RAG — Verifier Stage Metrics
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   ============================================================
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   Questions evaluated      : 1000
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   Exact Match (EM)         : 61.60%  (616/1000)
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   2026-05-07T16:57:23.968310552Z stdout F
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   Latency (p50 / p95)
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   retrieve               :   5353.6  /  5450.1
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   sample                 :      2.5  /  3.1
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   draft                  :   3733.0  /  6289.9
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   verify                 :     64.7  /  87.6
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   end-to-end             :   9180.0  /  11779.7
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   2026-05-07T16:57:23.970858895Z stdout F
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   Token throughput
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   drafter  in  (total)   :  2,249,775
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   drafter  out (total)   :    287,418
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   verifier in  (total)   :    595,632
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   drafter  in  (avg/q)   :     2249.8
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   drafter  out (avg/q)   :      287.4
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   verifier in  (avg/q)   :      595.6
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   Results saved → /gcs/speculative-rag-results-2026/verifier_output/no_opt.json
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   ============================================================
DEFAULT 2026-05-07 12:57:24 -0400       workerpool0-0   INFO | Verifier logic complete in 116.01s
DEFAULT 2026-05-07 12:57:25 -0400       workerpool0-0   INFO | SUCCESS: no_opt | Acc: 61.60% | p95 Draft: 6289.9ms
DEFAULT 2026-05-07 12:57:25 -0400       workerpool0-0   INFO | === E2E Pipeline Fully Completed ===
INFO    2026-05-07 12:57:55 -0400       service Tearing down training program.
INFO    2026-05-07 12:58:33 -0400       service Finished tearing down training program.
INFO    2026-05-07 12:58:33 -0400       service Job completed successfully.
```

Lessons Learned

What worked

vLLM continuous batching was the single biggest lever. Batching m=5 prompts in one scheduling window, not five serial calls, accounts for most of the ~2× draft latency speedup. (no_opt → vLLM: 3937 ms → 1932 ms)

k-means subset sampling on cached vectors is essentially free. <5 ms/query and gave us a measurable EM bump from draft diversity.

Profile-driven decisions. Nsight Systems showed that drafting was the bottleneck post retrieval. We stopped optimizing the wrong stages.

What didn't (or didn't matter)

NF4 solves the wrong problem in this setup-The 61% memory reduction is a structural benefit, but it comes at a 95% draft latency penalty due to per-token dequantization overhead in bitsandbytes.

Diminishing return in EM for m>10. p95 widens 45% while EM gains only ~4pp on 100 samples, suggesting the A100 stops fitting all prompts in one scheduling window.

Logprob scoring is fragile. Prompt-logprob vs. generated-token-logprob is not the same number; we added a single extra forward pass for correctness.

21M-passage index build is a 15 hour fixed cost. Worth doing once; cache on GCS, never rebuild.

FAISS-GPU was impossible. vLLM holds ~34GB, the 64GB passage index can't co-reside. Retrieval was done brute-force with CPU IndexFlatIP on 21m index at 5300ms/query.

Next Steps

Mixtral-8×7B Verifier on a multi-A100 / H100 node with tensor parallelism - measure whether a stronger verifier moves the EM ceiling without breaking the latency budget.

LoRA fine-tune the Drafter on 2k–10k high-quality {q, S, rationale, answer} traces distilled from the verifier - closes the gap between zero-shot rationale style and what the verifier prefers.

Add PubHealth as a second benchmark - distribution shift from open-domain trivia to claim verification stresses subset diversity differently. Same harness, swap the dataset loader.

FP8 (TransformerEngine) Drafter on H100 - replace NF4 with hardware-native FP8 to recover the quantization path that NF4-vs-vLLM closed off.

Speculative-decoding inside the Drafter - a small (1B) speculator behind Mistral-7B to compound the speedup at draft generation time.

Teamwork & Contributions

Member	Primary contributions	Tools / artifacts
Rupeet Kaur rk3408	Built full drafter pipeline from retrieving-sampling-drafting with final verifier inputs; modified sampling pipeline for KV cache retrieval, modified verifier pipeline to include profiling-vLLM, Pytorch profiler; modified fetcher.py for data integration in standard-rag, built entire standard-rag(generation to retrieval), built speculative rag tests-without opt, with nf4,int8, vLLM, m sweep, k sweep, profiling-pytorch profiler and nsys Final report:Experiment analysis,before-after optimization & profiling analysis Wrote **Substack Blog Post**	Drafter_pipeline.py, batched_drafter.py, vllm_.py,daft_output.py, run_tests.py, Check_verifier.py, biofetcher.py, standard-rag.py, retrieve_with_embeddings() in retriever.py Substack blog evidence
Soyoon Park sp4412	WanDB project and GitHub repository creation and management; Built dataset embedding (21m); Full baseline RAG test (~8k samples); Implemented and integrated the verifier logic to Speculative RAG; Speculative RAG tests: no_opt (1k samples), nf4, sweep on m={5,10,15,20}; Final Report: Abstract, Introduction, Discussion–limitation, Conclusion	Verifier.py code for Speculative RAG Wrapper for full Speculative RAG testing pipeline: Dockerfile, Makefile, submit.sh, e2e_eval.py
Hsuan-Ting Lin hl3930	Implemented and debugged Speculative RAG sampling logic, including multi-perspective subset construction from retrieved documents and integration with drafter inputs; ran and debugged no-opt and vLLM experiments on 100-sample tests; ran 1000-sample Speculative RAG tests; prepared profiler visualizations and performance analysis for no-opt, vLLM, and Standard RAG; analyzed latency, memory, throughput, and profiling results for presentation; wrote final report literature review and methodology sections.	Multi-perspective sampling code; Speculative RAG no_opt / vLLM result JSONs; PyTorch Profiler and Nsys trace screenshots; results/profiling slides.
Alexandar Vassilev av3341	Implemented Standard RAG baseline pipeline and GCP/Vertex evaluation setup.	standard-rag codebase; FAISS/Contriever retrieval; vLLM baseline; Standard RAG result JSON.

Thank you

Questions?

GitHub	github.com/soyoon-cu/speculative-rag
Paper	Wang et al., *Speculative RAG*, ICLR 2025
Profiles	nsys traces · ./nsys_traces/a100_vllm_m5_k2.nsys-rep
Tracking	tensorboard --logdir=./profiler_traces
Blog Post	https://rupeetkaur.substack.com/p/rebuilding-speculative-rag-implementation?r=8dild8

Backup — Experimental Setup

Backup slide. Use for Q&A. Add more backup slides as needed.

Setting	Standard RAG (baseline)	Speculative RAG (optimized)
Drafter / generator	Mistral-7B-Instruct v0.1, FP16	Mistral-7B-Instruct v0.1, FP16, vLLM PagedAttention
Verifier	—	Mistral-7B-Instruct v0.1
Retrieval	Contriever-MSMARCO + FAISS top-10 concat	Same retriever; top-n=10 → m=5 k-means subsets, k=2 docs each
Decoding	Greedy, T=0, max_new_tokens=300	Greedy, T=0, max_new_tokens=300, batched (m prompts / scheduler tick)
Quantization	None (FP16)	None for vLLM headline; NF4 / INT8 measured separately on HF path
Hardware	1× NVIDIA A100 80 GB · GCP Vertex AI	1× NVIDIA A100 80 GB · GCP Vertex AI (identical container)
Software stack	vllm/vllm-openai:v0.4.3, PyTorch 2.3.1, CUDA 12.1	+ vLLM continuous batching, NVTX, bitsandbytes 0.43.1 (HF path)
Eval set	TriviaQA validation, 1000 examples (containment EM)	TriviaQA validation, 100 and 1000 examples (containment EM)